

第 13 章

使用 BIND 提供域名解析服务

本章讲解了如下内容：

- DNS 域名解析服务；
- 安装 bind 服务程序；
- 部署从服务器；
- 安全的加密传输；
- 部署缓存服务器；
- 分离解析技术。

本章讲解了 DNS 域名解析服务的原理以及作用，介绍了域名查询功能中正向解析与反向解析的作用，并通过实验的方式演示了如何在 DNS 主服务器上部署正、反解析工作模式，以便让大家深刻体会到 DNS 域名查询的便利以及强大。

本章还介绍了如何部署 DNS 从服务器以及 DNS 缓存服务器来提升用户的域名查询体验，以及如何使用 chroot 牢笼机制插件来保障 bind 服务程序的可靠性，并向大家演示如何在主服务器与从服务器之间部署 TSIG 密钥加密功能，来进一步保障迭代查询中数据的安全性。最后，本章还从实战层面讲解了 DNS 分离解析技术，让来自不同国家、不同地区的用户都能获得最优的网站访问体验。

相信大家在学完本章内容之后，一定会对 bind 服务程序有更深入的了解和认识，并能深刻地体会到作为互联网基础设施中重要一环的 DNS 域名解析服务，在互联网中所承担的重要角色和发挥的重要作用。

13.1 DNS 域名解析服务

相较于由数字构成的 IP 地址，域名更容易被理解和记忆，所以我们通常更习惯通过域名的方式来访问网络中的资源。但是，网络中的计算机之间只能基于 IP 地址来相互识别对方的身份，而且要想在互联网中传输数据，也必须基于外网的 IP 地址来完成。

为了降低用户访问网络资源的门槛，DNS（Domain Name System，域名系统）技术应运而生。这是一项用于管理和解析域名与 IP 地址对应关系的技术，简单来说，就是能够接受用户输入的域名或 IP 地址，然后自动查找与之匹配（或者说具有映射关系）的 IP 地址或域名，即将域名解析为 IP 地址（正向解析），或将 IP 地址解析为域名（反向解析）。这样一来，我们只需要在浏览器中输入域名就能打开想要访问的网站了。DNS 域名解析技术的正向解析也是我们最常使用的一种工作模式。

鉴于互联网中的域名和 IP 地址对应关系数据库太过庞大，DNS 域名解析服务采用了类似目录树的层次结构来记录域名与 IP 地址之间的对应关系，从而形成了一个分布式的数据库系统，如图 13-1 所示。

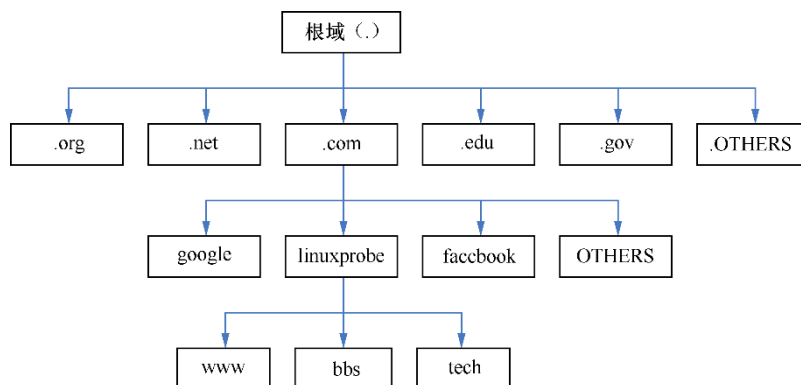


图 13-1 DNS 域名解析服务采用的目录树层次结构

域名后缀一般分为国际域名和国内域名。原则上讲，域名后缀都有严格的定义，但在实际使用时可以不必严格遵守。目前最常见的域名后缀有 .com（商业组织）、.org（非营利组织）、.gov（政府部门）、.net（网络服务商）、.edu（教研机构）、.pub（公共大众）、.cn（中国国家顶级域名）等。

当今世界的信息化程度越来越高，大数据、云计算、物联网、人工智能等新技术不断涌现，全球网民的数量据说也超过了 35 亿，而且每年还在以 10% 的速度迅速增长。这些因素导致互联网中的域名数量进一步激增，被访问的频率也进一步加大。假设全球网民每人每天只访问一个网站域名，而且只访问一次，也会产生 35 亿次的查询请求，如此庞大的请求数量肯定无法被某一台服务器全部处理掉。DNS 技术作为互联网基础设施中重要的一环，为了为网民提供不间断、稳定且快速的域名查询服务，保证互联网的正常运转，提供了下面三种类型的服务器。

- **主服务器：**在特定区域内具有唯一性，负责维护该区域内的域名与 IP 地址之间的对应关系。
- **从服务器：**从主服务器中获得域名与 IP 地址的对应关系并进行维护，以防主服务器宕机等情况。
- **缓存服务器：**通过向其他域名解析服务器查询获得域名与 IP 地址的对应关系，并将经常查询的域名信息保存到服务器本地，以此来提高重复查询时的效率。

简单来说，主服务器是用于管理域名和 IP 地址对应关系的真正服务器，从服务器帮助主服务器“打下手”，分散部署在各个国家、省市或地区，以便让用户就近查询域名，从而减轻主服务器的负载压力。缓存服务器不太常用，一般部署在企业内网的网关位置，用于加速用户的域名查询请求。

DNS 域名解析服务采用分布式的数据库结构来存放海量的“区域数据”信息，在执行用户发起的域名查询请求时，具有递归查询和迭代查询两种方式。所谓递归查询，是指 DNS 服务

器在收到用户发起的请求时，必须向用户返回一个准确的查询结果。如果 DNS 服务器本地没有存储与之对应的信息，则该服务器需要询问其他服务器，并将返回的查询结果提交给用户。而迭代查询则是指，DNS 服务器在收到用户发起的请求时，并不直接回复查询结果，而是告诉另一台 DNS 服务器的地址，用户再向这台 DNS 服务器提交请求，这样依次反复，直到返回查询结果。

由此可见，当用户向就近的一台 DNS 服务器发起对某个域名的查询请求之后（这里以 `www.linuxprobe.com` 为例），其查询流程大致如图 13-2 所示。

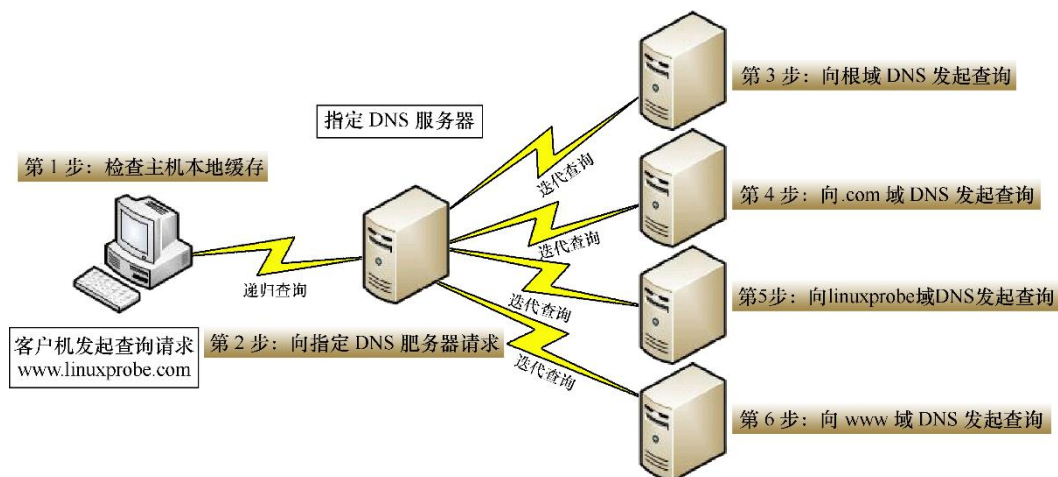


图 13-2 向 DNS 服务器发起域名查询请求的流程

当用户向网络指定的 DNS 服务器发起一个域名请求时，通常情况下会有本地由此 DNS 服务器向上级的 DNS 服务器发送迭代查询请求；如果该 DNS 服务器没有要查询的信息，则会进一步向上级 DNS 服务器发送迭代查询请求，直到获得准确的查询结果为止。其中最高级、最权威的根 DNS 服务器总共有 13 台，分布在世界各地，其管理单位、具体的地理位置，以及 IP 地址如表 13-1 所示。

表 13-1 13 台根 DNS 服务器的具体信息

名称	管理单位	地理位置	IP 地址
A	INTERNIC.NET	美国弗吉尼亚州	198.41.0.4
B	美国信息科学研究所	美国加利福尼亚州	128.9.0.107
C	PSINet 公司	美国弗吉尼亚州	192.33.4.12
D	马里兰大学	美国马里兰州	128.8.10.90
E	美国航空航天管理局	美国加利福尼亚州	192.203.230.10
F	因特网软件联盟	美国加利福尼亚州	192.5.5.241
G	美国国防部网络信息中心	美国弗吉尼亚州	192.112.36.4
H	美国陆军研究所	美国马里兰州	128.63.2.53
I	Autonomica 公司	瑞典斯德哥尔摩	192.36.148.17
J	VeriSign 公司	美国弗吉尼亚州	192.58.128.30
K	RIPE NCC	英国伦敦	193.0.14.129

L	IANA	美国弗吉尼亚州	199.7.83.42
M	WIDE Project	日本东京	202.12.27.33

13.2 安装 bind 服务程序

BIND (Berkeley Internet Name Domain, 伯克利因特网名称域) 服务是全球范围内使用最广泛、最安全可靠且高效的域名解析服务程序。DNS 域名解析服务作为互联网基础设施服务, 其责任之重可想而知, 因此建议大家在生产环境中安装部署 bind 服务程序时加上 chroot (俗称牢笼机制) 扩展包, 以便有效地限制 bind 服务程序仅能对自身的配置文件进行操作, 以确保整个服务器的安全。

```
[root@linuxprobe ~]# yum install bind-chroot
Loaded plugins: langpacks, product-id, subscription-manager
.....省略部分输出信息.....
Installing:
  bind-chroot x86_64 32:9.9.4-14.el7 rhel 81 k
Installing for dependencies:
  bind x86_64 32:9.9.4-14.el7 rhel 1.8 M
Transaction Summary
=====
Install 1 Package (+1 Dependent package)
Total download size: 1.8 M
Installed size: 4.3 M
Is this ok [y/d/N]: y
Downloading packages:
-----
Total 28 MB/s | 1.8 MB 00:00
Running transaction check
Running transaction test
Transaction test succeeded
Running transaction
  Installing : 32:bind-9.9.4-14.el7.x86_64 1/2
  Installing : 32:bind-chroot-9.9.4-14.el7.x86_64 2/2
  Verifying : 32:bind-9.9.4-14.el7.x86_64 1/2
  Verifying : 32:bind-chroot-9.9.4-14.el7.x86_64 2/2
Installed:
  bind-chroot.x86_64 32:9.9.4-14.el7
Dependency Installed:
  bind.x86_64 32:9.9.4-14.el7
Complete!
```

bind 服务程序的配置并不简单, 因为要想为用户提供健全的 DNS 查询服务, 要在本地保存相关的域名数据库, 而如果把所有域名和 IP 地址的对应关系都写入到某个配置文件中, 估计要有上千万条的参数, 这样既不利于程序的执行效率, 也不方便日后的修改和维护。因此在 bind 服务程序中有下面这三个比较关键的文件。

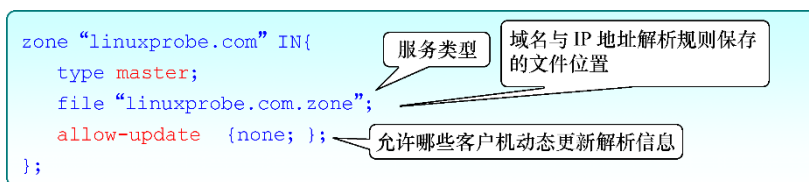
- 主配置文件 (/etc/named.conf): 只有 58 行, 而且在去除注释信息和空行之后, 实际有效的参数仅有 30 行左右, 这些参数用来定义 bind 服务程序的运行。
- 区域配置文件 (/etc/named.rfc1912.zones): 用来保存域名和 IP 地址对应关系的所在位置。类似于图书的目录, 对应着每个域和相应 IP 地址所在的具体位置, 当需要查看或修改时, 可根据这个位置找到相关文件。
- 数据配置文件目录 (/var/named): 该目录用来保存域名和 IP 地址真实对应关系的数据配置文件。

在 Linux 系统中，bind 服务程序的名称为 **named**。首先需要在 `/etc` 目录中找到该服务程序的主配置文件，然后把第 11 行和第 17 行的地址均修改为 **any**，分别表示服务器上的所有 IP 地址均可提供 DNS 域名解析服务，以及允许所有人对本服务器发送 DNS 查询请求。这两个地方一定要修改准确。

```
[root@linuxprobe ~]# vim /etc/named.conf
1 //
2 // named.conf
3 //
4 // Provided by Red Hat bind package to configure the ISC BIND named(8) DNS
5 // server as a caching only nameserver (as a localhost DNS resolver only).
6 //
7 // See /usr/share/doc/bind*/sample/ for example named configuration files.
8 //
9
10 options {
11 listen-on port 53 { any; };
12 listen-on-v6 port 53 { ::1; };
13 directory "/var/named";
14 dump-file "/var/named/data/cache_dump.db";
15 statistics-file "/var/named/data/named_stats.txt";
16 memstatistics-file "/var/named/data/named_mem_stats.txt";
17 allow-query { any; };
18
19 /*
20 - If you are building an AUTHORITATIVE DNS server, do NOT enable recursion.
21 - If you are building a RECURSIVE (caching) DNS server, you need to enable
22 recursion.
23 - If your recursive DNS server has a public IP address, you MUST enable access
24 control to limit queries to your legitimate users. Failing to do so will
25 cause your server to become part of large scale DNS amplification
26 attacks. Implementing BCP38 within your network would greatly
27 reduce such attack surface
28 */
29 recursion yes;
30
31 dnssec-enable yes;
32 dnssec-validation yes;
33 dnssec-lookaside auto;
34
35 /* Path to ISC DLV key */
36 bindkeys-file "/etc/named.iscdlv.key";
37
38 managed-keys-directory "/var/named/dynamic";
39
40 pid-file "/run/named/named.pid";
41 session-keyfile "/run/named/session.key";
42 };
43
44 logging {
45 channel default_debug {
```

```
46 file "data/named.run";
47 severity dynamic;
48 };
49 };
50
51 zone "." IN {
52 type hint;
53 file "named.ca";
54 };
55
56 include "/etc/named.rfc1912.zones";
57 include "/etc/named.root.key";
58
```

如前所述，bind 服务程序的区域配置文件（/etc/named.rfc1912.zones）用来保存域名和 IP 地址对应关系的所在位置。在这个文件中，定义了域名与 IP 地址解析规则保存的文件位置以及服务类型等内容，而没有包含具体的域名、IP 地址对应关系等信息。服务类型有三种，分别为 hint（根区域）、master（主区域）、slave（辅助区域），其中常用的 master 和 slave 指的就是主服务器和从服务器。将域名解析为 IP 地址的正向解析参数和将 IP 地址解析为域名的反向解析参数分别如图 13-3 和图 13-4 所示。



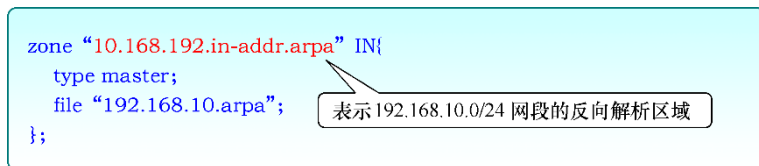
```
zone "linuxprobe.com" IN{
    type master;
    file "linuxprobe.com.zone";
    allow-update {none;};
};
```

服务类型

域名与 IP 地址解析规则保存的文件位置

允许哪些客户端动态更新解析信息

图 13-3 正向解析参数



```
zone "10.168.192.in-addr.arpa" IN{
    type master;
    file "192.168.10.arpa";
};
```

表示 192.168.10.0/24 网段的反向解析区域

图 13-4 反向解析参数

下面的实验中会分别修改 bind 服务程序的主配置文件、区域配置文件与数据配置文件。如果在实验中遇到了 bind 服务程序启动失败的情况，而您认为这是由于参数写错而导致的，则可以执行 named-checkconf 命令和 named-checkzone 命令，分别检查主配置文件与数据配置文件中语法或参数的错误。

13.2.1 正向解析实验

在 DNS 域名解析服务中，正向解析是指根据域名（主机名）查找到对应的 IP 地址。也就是说，当用户输入了一个域名后，bind 服务程序会自动进行查找，并将匹配到的 IP 地址返回给用户。这也是最常用的 DNS 工作模式。

第 1 步：编辑区域配置文件。该文件中默认已经有了一些无关紧要的解析参数，旨在让

用户有一个参考。我们可以将下面的参数添加到区域配置文件的最下面，当然，也可以将该文件中的原有信息全部清空，而只保留自己的域名解析信息：

```
[root@linuxprobe ~]# vim /etc/named.rfc1912.zones
zone "linuxprobe.com" IN {
    type master;
    file "linuxprobe.com.zone";
    allow-update {none;};
};
```

第 2 步：编辑数据配置文件。我们可以从/var/named 目录中复制一份正向解析的模板文件（named.localhost），然后把域名和 IP 地址的对应数据填写数据配置文件中并保存。在复制时记得加上-a 参数，这可以保留原始文件的所有者、所属组、权限属性等信息，以便让 bind 服务程序顺利读取文件内容：

```
[root@linuxprobe ~]# cd /var/named/
[root@linuxprobe named]# ls -al named.localhost
-rw-r-----. 1 root named 152 Jun 21 2007 named.localhost
[root@linuxprobe named]# cp -a named.localhost linuxprobe.com.zone
```

编辑数据配置文件。在保存并退出后文件后记得重启 named 服务程序，让新的解析数据生效。考虑到正向解析文件中的参数较多，而且相对都比较高重要，刘遑老师在每个参数后面都作了简要的说明。

```
[root@linuxprobe named]# vim linuxprobe.com.zone
[root@linuxprobe named]# systemctl restart named
```

\$TTL 1D	#生存周期为 1 天			
@	IN SOA	linuxprobe.com.	root.linuxprobe.com.	(
	#授权信息 开始	#DNS 区域的 地址	#域名管理员的邮箱（不要用@符号）	
			0;serial	#更新序列号
			1D;refresh	#更新时间
			1H;retry	#重试延时
			1W;expire	#失效时间
			3H);minimum	#无效解析记录的 缓存时间
	NS	ns.linuxprobe.com.		#域名服务器记录
ns	IN A	192.168.10.10		#地址记录（ ns.linuxprobe.com. ）
	IN MX 10	mail.linuxprobe.com.		#邮箱交换记录
mail	IN A	192.168.10.10		#地址记录（ mail.linuxprobe.com. ）
www	IN A	192.168.10.10		#地址记录（ www.linuxprobe.com. ）
bbs	IN A	192.168.10.20		#地址记录（ bbs.linuxprobe.com. ）

第 3 步：检验解析结果。为了检验解析结果，一定要先把 Linux 系统网卡中的 DNS 地址参数修改成本机 IP 地址，这样就可以使用由本机提供的 DNS 查询服务了。nslookup 命令用于检测能否从 DNS 服务器中查询到域名与 IP 地址的解析记录，进而更准确地检验 DNS 服务器是否已经能够为用户提供服务。

```
[root@linuxprobe ~]# systemctl restart network
[root@linuxprobe ~]# nslookup
> www.linuxprobe.com
Server: 127.0.0.1
Address: 127.0.0.1#53
Name: www.linuxprobe.com
Address: 192.168.10.10
> bbs.linuxprobe.com
Server: 127.0.0.1
Address: 127.0.0.1#53
Name: bbs.linuxprobe.com
Address: 192.168.10.20
```

13.2.2 反向解析实验

在 DNS 域名解析服务中，反向解析的作用是将用户提交的 IP 地址解析为对应的域名信息，它一般用于对某个 IP 地址上绑定的所有域名进行整体屏蔽，屏蔽由某些域名发送的垃圾邮件。它也可以针对某个 IP 地址进行反向解析，大致判断出有多少个网站运行在上面。当购买虚拟主机时，可以使用这一功能验证虚拟主机提供商是否有严重的超售问题。

第 1 步：编辑区域配置文件。在编辑该文件时，除了不要写错格式之外，还需要记住此处定义的数据配置文件名称，因为一会儿还需要在/var/named 目录中建立与其对应的同名文件。反向解析是把 IP 地址解析成域名格式，因此在定义 zone（区域）时应该要把 IP 地址反写，比如原来是 192.168.10.0，反写后应该就是 10.168.192，而且只需写出 IP 地址的网络位即可。把下列参数添加至正向解析参数的后面。

```
[root@linuxprobe ~]# vim /etc/named.rfc1912.zones
zone "linuxprobe.com" IN {
    type master;
    file "linuxprobe.com.zone";
    allow-update {none;};
};
zone "10.168.192.in-addr.arpa" IN {
    type master;
    file "192.168.10.arpa";
};
```

第 2 步：编辑数据配置文件。首先从/var/named 目录中复制一份反向解析的模板文件（named.loopback），然后把下面的参数填写到文件中。其中，IP 地址仅需要写主机位，如图 13-5 所示。

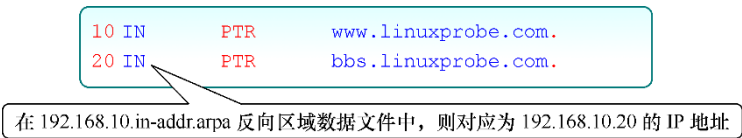


图 13-5 反向解析文件中 IP 地址参数规范

```
[root@linuxprobe named]# cp -a named.loopback 192.168.10.arpa
[root@linuxprobe named]# vim 192.168.10.arpa
[root@linuxprobe named]# systemctl restart named
```

\$TTL 1D			
@	IN SOA	linuxprobe.com.	root.linuxprobe.com. (0;serial 1D;refresh 1H;retry 1W;expire 3H);minimum
NS		ns.linuxprobe.com.	
ns	A	192.168.10.10	
10	PTR	ns.linuxprobe.com.	#PTR 为指针记录，仅用于反向解析
10	PTR	mail.linuxprobe.com.	
10	PTR	www.linuxprobe.com.	
20	PTR	bbs.linuxprobe.com.	

第 3 步：检验解析结果。在前面的正向解析实验中，已经把系统网卡中的 DNS 地址参数修改成了本机 IP 地址，因此可以直接使用 nslookup 命令来检验解析结果，仅需输入 IP 地址即可查询到对应的域名信息。

```
[root@linuxprobe ~]# nslookup
> 192.168.10.10
Server: 127.0.0.1
Address: 127.0.0.1#53
10.10.168.192.in-addr.arpa name = ns.linuxprobe.com.
10.10.168.192.in-addr.arpa name = www.linuxprobe.com.
10.10.168.192.in-addr.arpa name = mail.linuxprobe.com.
> 192.168.10.20
Server: 127.0.0.1
Address: 127.0.0.1#53
20.10.168.192.in-addr.arpa name = bbs.linuxprobe.com.
```

13.3 部署从服务器

作为重要的互联网基础设施服务，保证 DNS 域名解析服务的正常运转至关重要，只有这样才能提供稳定、快速且不间断的域名查询服务。在 DNS 域名解析服务中，从服务器可以从主服务器上获取指定的区域数据文件，从而起到备份解析记录与负载均衡的作用，因此通过部署从服务器可以减轻主服务器的负载压力，还可以提升用户的查询效率。

在本实验中，主服务器与从服务器分别使用的操作系统和 IP 地址如表 13-2 所示

表 13-2 主服务器与从服务器分别使用的操作系统与 IP 地址信息

主机名称	操作系统	IP 地址
主服务器	RHEL 7	192.168.10.10
从服务器	RHEL 7	192.168.10.20

第 1 步：在主服务器的区域配置文件中允许该从服务器的更新请求，即修改 `allow-update` {允许更新区域信息的主机地址;}参数，然后重启主服务器的 DNS 服务程序。

```
[root@linuxprobe ~]# vim /etc/named.rfc1912.zones
zone "linuxprobe.com" IN {
    type master;
    file "linuxprobe.com.zone";
    allow-update { 192.168.10.20; };
};
zone "10.168.192.in-addr.arpa" IN {
    type master;
    file "192.168.10.arpa";
    allow-update { 192.168.10.20; };
};
[root@linuxprobe ~]# systemctl restart named
```

第 2 步：在从服务器中填写主服务器的 IP 地址与要抓取的区域信息，然后重启服务。注意此时的服务类型应该是 `slave`（从），而不再是 `master`（主）。`masters` 参数后面应该为主服务器的 IP 地址，而且 `file` 参数后面定义的是同步数据配置文件后要保存到的位置，稍后可以在该目录内看到同步的文件。

```
[root@linuxprobe ~]# vim /etc/named.rfc1912.zones
zone "linuxprobe.com" IN {
    type slave;
    masters { 192.168.10.10; };
    file "slaves/linuxprobe.com.zone";
};
zone "10.168.192.in-addr.arpa" IN {
    type slave;
    masters { 192.168.10.10; };
    file "slaves/192.168.10.arpa";
};
[root@linuxprobe ~]# systemctl restart named
```

第 3 步：检验解析结果。当从服务器的 DNS 服务程序在重启后，一般就已经自动从主服

务器上同步了数据配置文件，而且该文件默认会放置在区域配置文件中定义的目录位置中。随后修改从服务器的网络参数，把 DNS 地址参数修改成 192.168.10.20，这样即可使用从服务器自身提供的 DNS 域名解析服务。最后就可以使用 nslookup 命令顺利看到解析结果了。

```
[root@linuxprobe ~]# cd /var/named/slaves
[root@linuxprobe slaves]# ls
192.168.10.arpa linuxprobe.com.zone
[root@linuxprobe slaves]# nslookup
> www.linuxprobe.com
Server: 192.168.10.20
Address: 192.168.10.20#53
Name: www.linuxprobe.com
Address: 192.168.10.10
> 192.168.10.10
Server: 192.168.10.20
Address: 192.168.10.20#53
10.10.168.192.in-addr.arpa name = www.linuxprobe.com.
10.10.168.192.in-addr.arpa name = ns.linuxprobe.com.
10.10.168.192.in-addr.arpa name = mail.linuxprobe.com.
```

13.4 安全的加密传输

前文反复提及，域名解析服务是互联网基础设施中重要的一环，几乎所有的网络应用都依赖于 DNS 才能正常运行。如果 DNS 服务发生故障，那么即便 Web 网站或电子邮件系统服务等都正常运行，用户也无法找到并使用它们了。

互联网中的绝大多数 DNS 服务器（超过 95%）都是基于 BIND 域名解析服务搭建的，而 bind 服务程序为了提供安全的解析服务，已经对 TSIG（RFC 2845）加密机制提供了支持。TSIG 主要是利用了密码编码的方式来保护区域信息的传输（Zone Transfer），即 TSIG 加密机制保证了 DNS 服务器之间传输域名区域信息的安全性。

接下来的实验依然使用了表 13-2 中的两台服务器。
书接上回。前面在从服务器上配妥 bind 服务程序并重启后，即可看到从主服务器中获取到的数据配置文件。

```
[root@linuxprobe ~]# ls -al /var/named/slaves/
total 12
drwxrwx---. 2 named named  54 Jun 7 16:02 .
drwxr-x---. 6 root  named 4096 Jun 7 15:58 ..
-rw-r--r---. 1 named named  432 Jun 7 16:02 192.168.10.arpa
-rw-r--r---. 1 named named  439 Jun 7 16:02 linuxprobe.com.zone
```

第 1 步：在主服务器中生成密钥。dnssec-keygen 命令用于生成安全的 DNS 服务密钥，其格式为“dnssec-keygen [参数]”，常用的参数以及作用如表 13-3 所示。

表 13-3 dnssec-keygen 命令的常用参数

参数	作用
-a	指定加密算法，包括 RSAMD5（RSA）、RSASHA1、DSA、NSEC3RSASHA1、NSEC3DSA

	等
-b	密钥长度（HMAC-MD5 的密钥长度在 1~512 位之间）
-n	密钥的类型（HOST 表示与主机相关）

使用下述命令生成一个主机名称为 `master-slave` 的 128 位 HMAC-MD5 算法的密钥文件。在执行该命令后默认会在当前目录中生成公钥和私钥文件，我们需要把私钥文件中 `Key` 参数后面的值记录下来，一会儿要将其写入传输配置文件中。

```
[root@linuxprobe ~]# dnssec-keygen -a HMAC-MD5 -b 128 -n HOST master-slave
Kmaster-slave.+157+46845
[root@linuxprobe ~]# ls -al Kmaster-slave.+157+46845.*
-rw-----. 1 root root 56 Jun 7 16:06 Kmaster-slave.+157+46845.key
-rw-----. 1 root root 165 Jun 7 16:06 Kmaster-slave.+157+46845.private
[root@linuxprobe ~]# cat Kmaster-slave.+157+46845.private
Private-key-format: v1.3
Algorithm: 157 (HMAC_MD5)
Key: 1XEEL3tG5DNLOw+1WHfE3Q==
Bits: AAA=
Created: 20170607080621
Publish: 20170607080621
Activate: 20170607080621
```

第 2 步：在主服务器中创建密钥验证文件。进入 `bind` 服务程序用于保存配置文件的目录，把刚刚生成的密钥名称、加密算法和私钥加密字符串按照下面格式写入到 `transfer.key` 传输配置文件中。为了安全起见，我们需要将文件的所属组修改成 `named`，并将文件权限设置得要小一点，然后把该文件做一个硬链接到 `/etc` 目录中。

```
[root@linuxprobe ~]# cd /var/named/chroot/etc/
[root@linuxprobe etc]# vim transfer.key
key "master-slave" {
    algorithm hmac-md5;
    secret "1XEEL3tG5DNLOw+1WHfE3Q==";
};
[root@linuxprobe ~]# chown root:named transfer.key
[root@linuxprobe ~]# chmod 640 transfer.key
[root@linuxprobe ~]# ln transfer.key /etc/transfer.key
```

第 3 步：开启并加载 `Bind` 服务的密钥验证功能。首先需要在主服务器的主配置文件中加载密钥验证文件，然后进行设置，使得只允许带有 `master-slave` 密钥认证的 `DNS` 服务器同步数据配置文件：

```
[root@linuxprobe ~]# vim /etc/named.conf
1 //
2 // named.conf
3 //
4 // Provided by Red Hat bind package to configure the ISC BIND named(8) DNS
5 // server as a caching only nameserver (as a localhost DNS resolver only).
6 //
7 // See /usr/share/doc/bind*/sample/ for example named configuration files.
```

```
8 //
9 include "/etc/transfer.key";
10 options {
11 listen-on port 53 { any; };
12 listen-on-v6 port 53 { ::1; };
13 directory "/var/named";
14 dump-file "/var/named/data/cache_dump.db";
15 statistics-file "/var/named/data/named_stats.txt";
16 memstatistics-file "/var/named/data/named_mem_stats.txt";
17 allow-query { any; };
18 allow-transfer { key master-slave; };
.....省略部分输出信息.....
[root@linuxprobe ~]# systemctl restart named
```

至此，DNS 主服务器的 TSIG 密钥加密传输功能就已经配置完成。此时清空 DNS 从服务器同步目录中所有的数据配置文件，然后再次重启 bind 服务程序，这时就已经不能像刚才那样自动获取到数据配置文件了。

```
[root@linuxprobe ~]# rm -rf /var/named/slaves/*
[root@linuxprobe ~]# systemctl restart named
[root@linuxprobe ~]# ls /var/named/slaves/
```

第 4 步：配置从服务器，使其支持密钥验证。配置 DNS 从服务器和主服务器的方法大致相同，都需要在 bind 服务程序的配置文件目录中创建密钥认证文件，并设置相应的权限，然后把该文件做一个硬链接到/etc 目录中。

```
[root@linuxprobe ~]# cd /var/named/chroot/etc
[root@linuxprobe etc]# vim transfer.key
key "master-slave" {
algorithm hmac-md5;
secret "1XEEL3tG5DNLOw+1WHfE3Q==";
};
[root@linuxprobe etc]# chown root:named transfer.key
[root@linuxprobe etc]# chmod 640 transfer.key
[root@linuxprobe etc]# ln transfer.key /etc/transfer.key
```

第 5 步：开启并加载从服务器的密钥验证功能。这一步的操作步骤也同样是在主配置文件中加载密钥认证文件，然后按照指定格式写上主服务器的 IP 地址和密钥名称。注意，密钥名称等参数位置不要太靠前，大约在第 43 行比较合适，否则 bind 服务程序会因为没有加载完预设参数而报错：

```
[root@linuxprobe etc]# vim /etc/named.conf
1 //
2 // named.conf
3 //
4 // Provided by Red Hat bind package to configure the ISC BIND named(8) DNS
5 // server as a caching only nameserver (as a localhost DNS resolver only).
6 //
7 // See /usr/share/doc/bind*/sample/ for example named configuration files.
8 //
```

```
9 include "/etc/transfer.key";
10 options {
11 listen-on port 53 { 127.0.0.1; };
12 listen-on-v6 port 53 { ::1; };
13 directory "/var/named";
14 dump-file "/var/named/data/cache_dump.db";
15 statistics-file "/var/named/data/named_stats.txt";
16 memstatistics-file "/var/named/data/named_mem_stats.txt";
17 allow-query { localhost; };
18
19 /*
20 - If you are building an AUTHORITATIVE DNS server, do NOT enable recursion.
21 - If you are building a RECURSIVE (caching) DNS server, you need to enable
22 recursion.
23 - If your recursive DNS server has a public IP address, you MUST enable access
24 control to limit queries to your legitimate users. Failing to do so will
25 cause your server to become part of large scale DNS amplification
26 attacks. Implementing BCP38 within your network would greatly
27 reduce such attack surface
28 */
29 recursion yes;
30
31 dnssec-enable yes;
32 dnssec-validation yes;
33 dnssec-lookaside auto;
34
35 /* Path to ISC DLV key */
36 bindkeys-file "/etc/named.iscdlv.key";
37
38 managed-keys-directory "/var/named/dynamic";
39
40 pid-file "/run/named/named.pid";
41 session-keyfile "/run/named/session.key";
42 };
43 server 192.168.10.10
44 {
45 keys { master-slave; };
46 };
47 logging {
48 channel default_debug {
49 file "data/named.run";
50 severity dynamic;
51 };
52 };
53
54 zone "." IN {
55 type hint;
56 file "named.ca";
57 };
58
59 include "/etc/named.rfc1912.zones";
60 include "/etc/named.root.key";
61
```

第 6 步：DNS 从服务器同步域名区域数据。现在，两台服务器的 bind 服务程序都已经配置妥当，并匹配到了相同的密钥认证文件。接下来在从服务器上重启 bind 服务程序，可以发现又能顺利地同步到数据配置文件了。

```
[root@linuxprobe ~]# systemctl restart named
[root@linuxprobe ~]# ls /var/named/slaves/
192.168.10.arpa  linuxprobe.com.zone
```

13.5 部署缓存服务器

DNS 缓存服务器（Caching DNS Server）是一种不负责域名数据维护的 DNS 服务器。简单来说，缓存服务器就是把用户经常使用到的域名与 IP 地址的解析记录保存在主机本地，从而提升下次解析的效率。DNS 缓存服务器一般用于经常访问某些固定站点而且对这些网站的访问速度有较高要求的企业内网中，但实际的应用并不广泛。而且，缓存服务器是否可以成功解析还与指定的上级 DNS 服务器的允许策略有关，因此当前仅需了解即可。

第 1 步：配置系统的双网卡参数。前面讲到，缓存服务器一般用于企业内网，旨在降低内网用户查询 DNS 的时间消耗。因此，为了更加贴近真实的网络环境，实现外网查询功能，我们需要在缓存服务器中再添加一块网卡，并按照表 13-4 所示的信息来配置出两台 Linux 虚拟机系统。而且，还需要在虚拟机软件中将新添加的网卡设置为“桥接模式”，然后设置成与物理设备相同的网络参数（此处需要大家按照物理设备真实的网络参数来配置，图 13-6 所示为以 DHCP 方式获取 IP 地址与网关等信息，重启网络服务后的效果如图 13-7 所示）。

表 13-4 用于配置 Linux 虚拟机系统所需的参数信息

主机名称	操作系统	IP 地址
缓存服务器	RHEL 7	网卡（外网）：根据物理设备的网络参数进行配置（通过 DHCP 或手动方式指定 IP 地址与网关等信息）
		网卡（内网）：192.168.10.10
客户端	RHEL 7	192.168.10.20

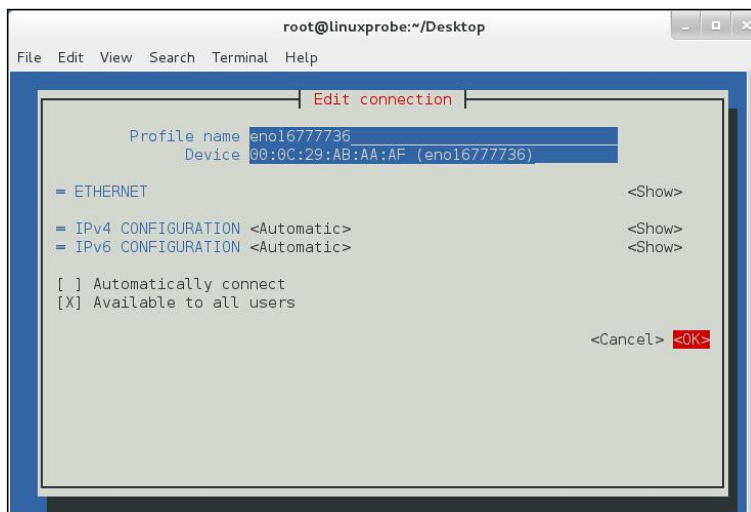


图 13-6 以 DHCP 方式获取网络参数



图 13-7 查看网卡的工作状态

第 2 步：在 bind 服务程序的主配置文件中添加缓存转发参数。在大约第 17 行处添加一行参数“forwarders { 上级 DNS 服务器地址; };”，上级 DNS 服务器地址指的是获取数据配置文件的服务器。考虑到查询速度、稳定性、安全性等因素，刘遄老师在这里使用的是北京市公共 DNS 服务器的地址 210.73.64.1。如果大家也使用该地址，请先测试是否可以 ping 通，以免导致 DNS 域名解析失败。

```
[root@linuxprobe ~]# vim /etc/named.conf
1 //
2 // named.conf
3 //
4 // Provided by Red Hat bind package to configure the ISC BIND named(8) DNS
5 // server as a caching only nameserver (as a localhost DNS resolver only).
```



```

6 //
7 // See /usr/share/doc/bind*/sample/ for example named configuration files.
8 //
9 options {
10 listen-on port 53 { any; };
11 listen-on-v6 port 53 { ::1; };
12 directory "/var/named";
13 dump-file "/var/named/data/cache_dump.db";
14 statistics-file "/var/named/data/named_stats.txt";
15 memstatistics-file "/var/named/data/named_mem_stats.txt";
16 allow-query { any; };
17 forwarders { 210.73.64.1; };
.....省略部分输出信息.....
[root@linuxprobe ~]# systemctl restart named

```

第3步：重启 DNS 服务，验证成果。把客户端主机的 DNS 服务器地址参数修改为 DNS 缓存服务器的 IP 地址 192.168.10.10，如图 13-8 所示。这样即可让客户端使用本地 DNS 缓存服务器提供的域名查询解析服务。

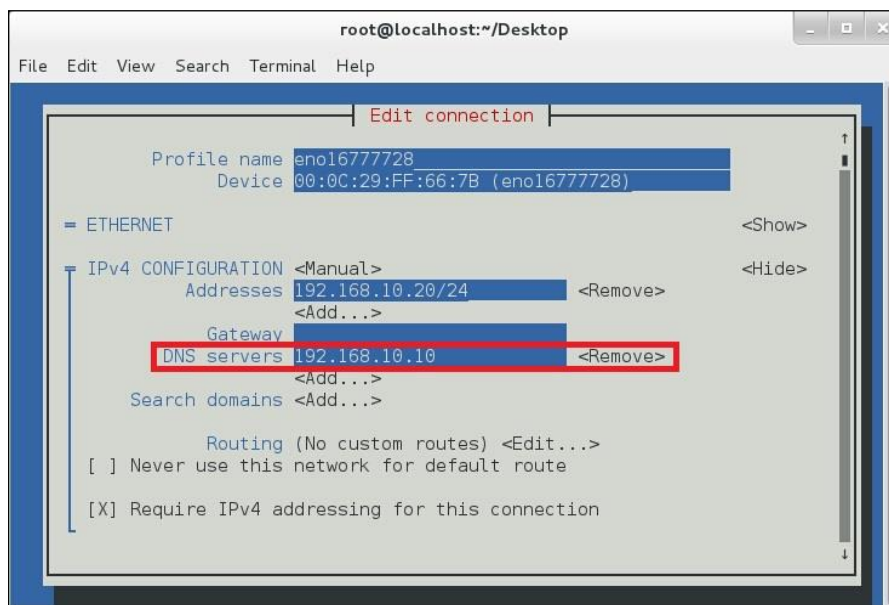


图 13-8 设置客户端主机的 DNS 服务器地址参数

在将客户端主机的网络参数设置妥当后重启网络服务，即可使用 `nslookup` 命令来验证实验结果（如果解析失败，请读者留意是否是上级 DNS 服务器选择的问题）。其中，`Server` 参数为域名解析记录提供的服务器地址，因此可见是由本地 DNS 缓存服务器提供的解析内容。

```

[root@linuxprobe ~]# nslookup
> www.linuxprobe.com
Server: 192.168.10.10
Address: 192.168.10.10#53

Non-authoritative answer:

```

```
Name: www.linuxprobe.com
Address: 113.207.76.73
Name: www.linuxprobe.com
Address: 116.211.121.154
> 8.8.8.8
Server: 192.168.10.10
Address: 192.168.10.10#53

Non-authoritative answer:
8.8.8.8.in-addr.arpa name = google-public-dns-a.google.com.
Authoritative answers can be found from:
in-addr.arpa nameserver = f.in-addr-servers.arpa.
in-addr.arpa nameserver = b.in-addr-servers.arpa.
in-addr.arpa nameserver = a.in-addr-servers.arpa.
in-addr.arpa nameserver = e.in-addr-servers.arpa.
in-addr.arpa nameserver = d.in-addr-servers.arpa.
in-addr.arpa nameserver = c.in-addr-servers.arpa.
a.in-addr-servers.arpa internet address = 199.212.0.73
a.in-addr-servers.arpa has AAAA address 2001:500:13::73
b.in-addr-servers.arpa internet address = 199.253.183.183
b.in-addr-servers.arpa has AAAA address 2001:500:87::87
c.in-addr-servers.arpa internet address = 196.216.169.10
c.in-addr-servers.arpa has AAAA address 2001:43f8:110::10
d.in-addr-servers.arpa internet address = 200.10.60.53
d.in-addr-servers.arpa has AAAA address 2001:13c7:7010::53
e.in-addr-servers.arpa internet address = 203.119.86.101
e.in-addr-servers.arpa has AAAA address 2001:dd8:6::101
f.in-addr-servers.arpa internet address = 193.0.9.1
f.in-addr-servers.arpa has AAAA address 2001:67c:e0::1
```

13.6 分离解析技术

现在，喜欢看我们这本《Linux 就该这么学》的海外读者越来越多，如果继续把本书配套的网站服务器（<http://www.linuxprobe.com>）架设在北京市的机房内，则海外读者的访问速度势必会很慢。可如果把服务器架设在美国那边的机房，也将增大国内读者的访问难度。

为了满足海内外读者的需求，外加刘遑老师不差钱，于是可以购买多台服务器并分别部署在全球各地，然后再使用 DNS 服务的分离解析功能，即可让位于不同地理范围内的读者通过访问相同的网址，而从不同的服务器获取到相同的数据。例如，我们可以按照表 13-5 所示，分别为处于北京的 DNS 服务器和处于美国的 DNS 服务器分配不同的 IP 地址，然后让国内读者在访问时自动匹配到北京的服务器，而让海外读者自动匹配到美国的服务器，如图 13-9 所示。

表 13-5 不同主机的操作系统与 IP 地址情况

主机名称	操作系统	IP 地址
DNS 服务器	RHEL 7	北京网络: 122.71.115.10
		美国网络: 106.185.25.10

国内读者	Windows 7	122.71.115.1
海外读者	Windows7	106.185.25.1

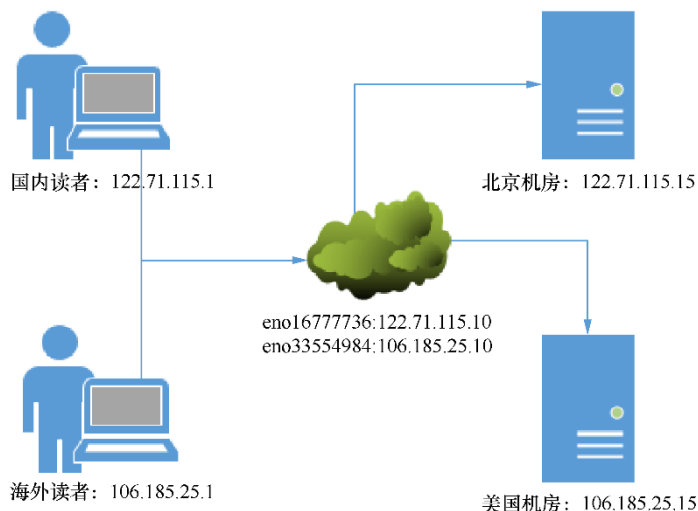


图 13-9 DNS 分离解析技术

为了解决海外读者访问 <http://www.Linuxprobe.com> 时的速度问题，刘遑老师已经在美国机房购买并架设好了相应的网站服务器，接下来需要手动部署 DNS 服务器并实现分离解析功能，以便让不同地理区域的读者在访问相同的域名时，能解析出不同的 IP 地址。

注：

建议大家将虚拟机还原到初始状态，并重新安装 bind 服务程序，以免多个实验之间相互产生冲突。

第 1 步：修改 bind 服务程序的主配置文件，把第 11 行的监听端口与第 17 行的允许查询主机修改为 any。由于配置的 DNS 分离解析功能与 DNS 根服务器配置参数有冲突，所以需要把第 51~54 行的根域信息删除。

```
[root@linuxprobe ~]# vim /etc/named.conf
.....省略部分输出信息.....
44 logging {
45   channel default_debug {
46     file "data/named.run";
47     severity dynamic;
48   };
49 };
50
51 zone "." IN {
52 type hint;
53 file "named.ca";
54 };
55
```

```
56 include "/etc/named.rfc1912.zones";
57 include "/etc/named.root.key";
58
.....省略部分输出信息.....
```

第 2 步：编辑区域配置文件。把区域配置文件中原有的数据清空，然后按照以下格式写入参数。首先使用 `acl` 参数分别定义两个变量名称（`china` 与 `american`），当下面需要匹配 IP 地址时只需写入变量名称即可，这样不仅容易阅读识别，而且也利于修改维护。这里的难点是理解 `view` 参数的作用。它的作用是通过判断用户的 IP 地址是中国的还是美国的，然后去分别加载不同的数据配置文件（`linuxprobe.com.china` 或 `linuxprobe.com.american`）。这样，当把相应的 IP 地址分别写入到数据配置文件后，即可实现 DNS 的分离解析功能。这样一来，当中国的用户访问 `linuxprobe.com` 域名时，便会按照 `linuxprobe.com.china` 数据配置文件内的 IP 地址找到对应的服务器。

```
[root@linuxprobe ~]# vim /etc/named.rfc1912.zones
1 acl "china" { 122.71.115.0/24; };
2 acl "american" { 106.185.25.0/24; };
3 view "china"{
4 match-clients { "china"; };
5 zone "linuxprobe.com" {
6 type master;
7 file "linuxprobe.com.china";
8 };
9 };
10 view "american" {
11 match-clients { "american"; };
12 zone "linuxprobe.com" {
13 type master;
14 file "linuxprobe.com.american";
15 };
16 };
```

第 3 步：建立数据配置文件。分别通过模板文件创建出两份不同名称的区域数据文件，其名称应与上面区域配置文件中的参数相对应。

```
[root@linuxprobe ~]# cd /var/named
[root@linuxprobe named]# cp -a named.localhost linuxprobe.com.china
[root@linuxprobe named]# cp -a named.localhost linuxprobe.com.american
[root@linuxprobe named]# vim linuxprobe.com.china
```

\$TTL	ID	#生存周期为 1 天		
@	IN SOA	linuxprobe.com.	root.linuxprobe.com	(

#授权信息开始	#DNS 区域的地 址	#域名管理员的邮箱（不要用@符 号）	
		0;serial	#更新序列号
		1D;refresh	#更新时间
		1H;retry	#重试延时
		1W;expire	#失效时间
		3H;minimum	#无效解析记录的 缓存时间
NS	ns.linuxprobe.com.		#域名服务器记录
ns	IN A	122.71.115.10	#地址记录（ ns.linuxprobe.com. ）
www	IN A	122.71.115.15	#地址记录（ www.linuxprobe.com. ）

```
[root@linuxprobe named]# vim linuxprobe.com.American
```

\$TTL 1D	#生存周期为 1 天			
@	IN SOA	linuxprobe.com.	root.linuxprobe.com	(
#授权信息开始	#DNS 区域的地 址	#域名管理员的邮箱（不要用@符 号）		
		0;serial		#更新序列号
		1D;refresh		#更新时间
		1H;retry		#重试延时
		1W;expire		#失效时间
		3H;minimum		#无效解析记录的 缓存时间
NS	ns.linuxprobe.com.			#域名服务器记录
ns	IN A	106.185.25.10		#地址记录（ ns.linuxprobe.com. ）
www	IN A	106.185.25.15		#地址记录（ www.linuxprobe.com. ）

第 4 步：重新启动 named 服务程序，验证结果。将客户端主机（Windows 系统或 Linux 系统均可）的 IP 地址分别设置为 122.71.115.1 与 106.185.25.1，将 DNS 地址分别设置为服务器主机的两个 IP 地址。这样，当尝试使用 nslookup 命令解析域名时就能清晰地看到解析结果，分别如图 13-10 与图 13-11 所示。

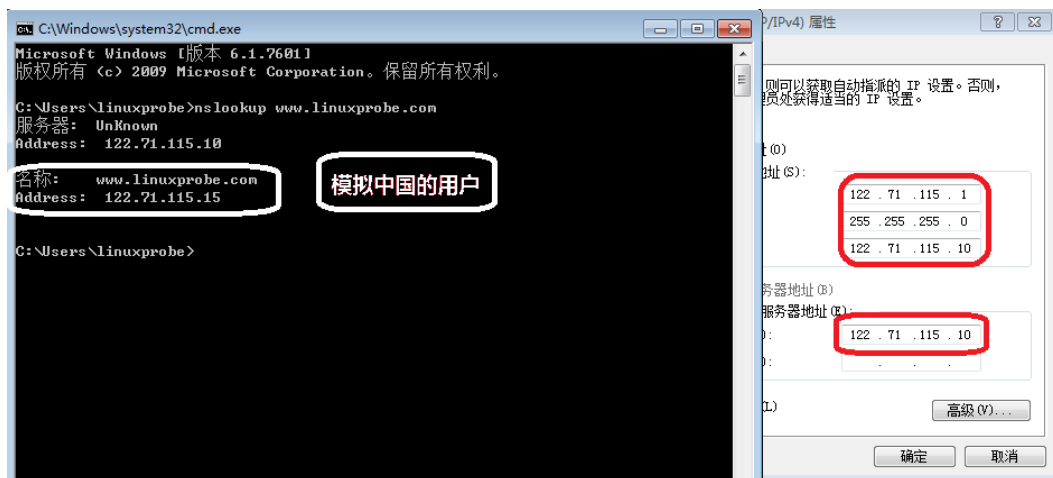


图 13-10 模拟中国用户的域名解析操作



图 13-11 模拟美国用户的域名解析

复习题

1. DNS 技术提供的三种类型的服务器分别是什么？

答：DNS 主服务器、DNS 从服务器与 DNS 缓存服务器。

2. DNS 服务器之间传输区域数据文件时，使用的是递归查询还是迭代查询？

答：DNS 服务器之间是迭代查询，用户与 DNS 服务器之间是递归查询。

3. 在 Linux 系统中使用 Bind 服务程序部署 DNS 服务时，为什么推荐安装 chroot 插件？

答：能有效地限制 Bind 服务程序仅能对自身的配置文件进行操作，以确保整个服务器的安全。

4. 在 DNS 服务中，正向解析和反向解析的作用是什么？

答：正向解析是将指定的域名转换为 IP 地址，而反向解析则是将 IP 地址转换为域名。正向解析模式更为常用。

5. 是否可以限制使用 DNS 域名解析服务的主机？如何限制？

答：是的，修改主配置文件中第 17 行的 allow-query 参数即可。

6. 部署 DNS 从服务器的作用是什么？

答：部署从服务器可以减轻主服务器的负载压力，还可以提升用户的查询效率。

7. 当用户与 DNS 服务器之间传输数据配置文件时，是否可以使用 TSIG 加密机制来确保文件内容不被篡改？

答：不能，TSIG 加密机制保障的是 DNS 服务器与 DNS 服务器之间迭代查询的安全。

8. 部署 DNS 缓存服务器的作用是什么？

答：DNS 缓存服务器把用户经常使用到的域名与 IP 地址的解析记录保存在主机本地，从而提升下次解析的效率。一般用于经常访问某些固定站点而且对这些网站的访问速度有较高要求的企业内网中，但实际的应用并不广泛。

9. DNS 分离解析技术的作用是什么？

答：可以让位于不同地理范围内的读者通过访问相同的网址，而从不同的服务器获取到相同的数据，以提升访问效率。